

# I2C DRIVER DEVELOPMENT

## STM32F407xx – Structured Lecture Notes

---

This document compiles six lectures covering the complete groundwork for building a bare-metal I2C peripheral driver for the STM32F407xx microcontroller — from initial file setup through clock configuration and clock stretching.

| # | Lecture                                            |
|---|----------------------------------------------------|
| 1 | Driver API Requirements & Initial Setup            |
| 2 | I2C Handle Structure & Configuration Structure     |
| 3 | Configuration Macros for User Options              |
| 4 | Creating API Prototypes                            |
| 5 | Introduction to I2C_Init() API                     |
| 6 | Serial Clock (SCL) Configuration using CR2 and CCR |
| 7 | I2C Protocol – Clock Stretching                    |

## LECTURE 1

# Driver API Requirements & Initial Setup

## Lecture Objective

In this lecture, the goal is to begin the I2C driver development by:

- Creating the required driver source and header files.
- Modifying the MCU-specific header file.
- Understanding the APIs that the driver should provide.
- Identifying the configuration options that the application user should be able to set.

## 1. Driver Files to Create

Inside the Drivers folder, create the following files:

```
Drivers/  
|  
+-- stm32f407xx_i2c_driver.c  
+-- stm32f407xx_i2c_driver.h
```

### *Purpose of Each File*

| File                     | Contains                                                                                                              |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------|
| stm32f407xx_i2c_driver.c | Driver function implementations, initialization code, transmission functions, reception functions, interrupt handlers |
| stm32f407xx_i2c_driver.h | Function declarations, macros, configuration structures, register bit definitions, user APIs                          |

## 2. Modify the MCU-Specific Header File

The device header file, **stm32f407xx.h**, must be updated with all I2C-related definitions. It should include:

- I2C base addresses
- Register definition structure
- Peripheral definition macros
- Clock enable/disable macros
- Register bit position macros

## 3. Driver APIs Required

The driver should expose several APIs to the application.

### ***A. Initialization API***

Used to initialize the I2C peripheral according to the user configuration.

```
I2C_Init();
```

#### **Responsibilities:**

- Configure SCL speed
- Configure own slave address
- Configure ACK
- Configure duty cycle

### ***B. Master Transmission API***

Used when MCU acts as Master and sends data.

```
I2C_MasterSendData();
```

#### **Master is responsible for:**

- Generating START condition
- Sending slave address
- Sending data bytes
- Generating STOP condition

### ***C. Master Reception API***

Used when MCU acts as Master and receives data.

```
I2C_MasterReceiveData();
```

#### **Master performs:**

- START generation
- Address phase
- Reading received bytes
- STOP generation

### ***D. Slave Transmission API***

Used when MCU acts as Slave.

```
I2C_SlaveSendData();
```

Unlike master, **slave does not:**

- Generate START
- Generate STOP
- Initiate communication

Slave only responds when addressed.

### ***E. Slave Reception API***

```
I2C_SlaveReceiveData();
```

Again, slave waits until the master initiates communication.

### ***F. Interrupt Handling APIs***

Interrupt support will also be provided. Examples:

```
I2C_IRQInterruptConfig()  
I2C_IRQPriorityConfig()  
I2C_IRQHandling()
```

Detailed implementation will be covered later.

### ***G. Other Generic Peripheral APIs***

Like previous drivers (GPIO, SPI), I2C also requires common APIs. Examples include:

- Peripheral Clock Enable
- Peripheral Clock Disable
- Peripheral Enable
- Peripheral Disable
- Peripheral Reset
- Flag Status APIs

### ***Summary of Driver APIs***

| API            | Purpose                  |
|----------------|--------------------------|
| Initialization | Configure I2C peripheral |
| Master TX      | Master sends data        |
| Master RX      | Master receives data     |
| Slave TX       | Slave transmits data     |
| Slave RX       | Slave receives data      |
| Interrupt APIs | Handle interrupts        |

| API                     | Purpose                         |
|-------------------------|---------------------------------|
| Clock Control APIs      | Enable/Disable peripheral clock |
| Peripheral Control APIs | Enable/Disable I2C              |

## 4. User Configurable Items

The driver should allow the application developer to configure important I2C parameters. These settings are stored inside a configuration structure.

### *Configurable Item 1 — SCL Speed*

The user specifies the desired serial clock frequency. Example values: 100 KHz or 400 KHz.

Configuration member: **I2C\_SCLSpeed** — Purpose: determines communication speed.

### *Configurable Item 2 — Device Own Address*

If the MCU operates as a slave, the application must specify its own address. Example: 0x68.

Configuration member: **I2C\_DeviceAddress** — Purpose: allows the slave to recognize when it is being addressed.

### *Configurable Item 3 — ACK Control*

By default, ACK generation is disabled. The user should decide whether ACK is Enabled or Disabled.

Configuration member: **I2C\_ACKControl** — Purpose: determines whether the peripheral automatically acknowledges received bytes.

### *Configurable Item 4 — Fast Mode Duty Cycle*

This applies only when SCL Speed > 100 KHz. This mode is called Fast Mode. In Fast Mode, different duty cycles are supported.

Configuration member: **I2C\_FMDutyCycle**. The detailed concept of duty cycle is covered later.

### *Complete Configuration Structure*

The configuration structure will approximately contain:

```
typedef struct
{
    uint32_t I2C_SCLSpeed;
    uint8_t I2C_DeviceAddress;
    uint8_t I2C_ACKControl;
    uint8_t I2C_FMDutyCycle;
}I2C_Config_t;
```

### *Summary of User Configuration Parameters*

| Member            | Purpose                 |
|-------------------|-------------------------|
| I2C_SCLSpeed      | Serial Clock Speed      |
| I2C_DeviceAddress | Slave Address           |
| I2C_ACKControl    | Enable/Disable ACK      |
| I2C_FMDutyCycle   | Duty Cycle in Fast Mode |

## 5. Fast Mode Overview

| Mode          | Speed                             |
|---------------|-----------------------------------|
| Standard Mode | Up to 100 KHz                     |
| Fast Mode     | Above 100 KHz (typically 400 KHz) |

Only Fast Mode requires duty-cycle configuration.

## 6. Modifications Required in stm32f407xx.h

The MCU-specific header file must include all I2C-related information.

### Step 1 — Add Base Address Macros

Define base addresses for I2C1, I2C2, I2C3.

```
#define I2C1_BASEADDR
#define I2C2_BASEADDR
#define I2C3_BASEADDR
```

### Step 2 — Create Register Definition Structure

Create a structure representing all I2C registers.

```
typedef struct
{
    volatile uint32_t CR1;
    volatile uint32_t CR2;
    volatile uint32_t OAR1;
    volatile uint32_t OAR2;
    volatile uint32_t DR;
    volatile uint32_t SR1;
    volatile uint32_t SR2;
    volatile uint32_t CCR;
    volatile uint32_t TRISE;
    volatile uint32_t FLTR;
} I2C_RegDef_t;
```

### Step 3 — Peripheral Definition Macros

Create peripheral pointers.

```
#define I2C1
#define I2C2
#define I2C3
```

#### Step 4 — Clock Enable/Disable Macros

Macros should enable I2C1 Clock, I2C2 Clock, I2C3 Clock. Also create corresponding disable macros.

#### Step 5 — Register Bit Position Macros

Create macros for important register bits.

**Required registers** — create bit position definitions for: CR1, CR2, SR1, SR2, CCR.

**Registers not required** (at this stage): OAR1, OAR2, TRISE.

| Register | Required?              |
|----------|------------------------|
| CR1      | Yes                    |
| CR2      | Yes                    |
| SR1      | Yes                    |
| SR2      | Yes                    |
| CCR      | Yes                    |
| OAR1     | No                     |
| OAR2     | No                     |
| TRISE    | Not required currently |

## Files After Completing the Exercise

```
Project
|
+-- Drivers
|   +-- stm32f407xx_i2c_driver.c
|   +-- stm32f407xx_i2c_driver.h
|
+-- stm32f407xx.h
    |
    +-- Base Address Macros
    +-- Register Structure
    +-- Peripheral Macros
    +-- Clock Enable Macros
    +-- Register Bit Macros
```

## Lecture Workflow

```
graph TD; A[Create Driver Files] --> B[Modify Device Header]; B --> C[Add I2C Register Structure]; C --> D[Add Base Address Macros]; D --> E[Add Peripheral Macros]; E --> F[Add Clock Macros]; F --> G[Add Register Bit Definitions]; G --> H[Design Driver APIs]; H --> I[Create User Configuration Structure];
```

## KEY TAKEAWAYS

- Create `stm32f407xx_i2c_driver.c` and `stm32f407xx_i2c_driver.h`.
- Update `stm32f407xx.h` with all I2C-related definitions.
- The driver should provide separate APIs for Master TX, Master RX, Slave TX, and Slave RX.
- Master handles START, Address, and STOP conditions; the slave only responds when addressed.
- Four user-configurable parameters: SCL Speed, Device (Own) Address, ACK Control, Fast Mode Duty Cycle.
- Add register bit definitions only for CR1, CR2, SR1, SR2, and CCR at this stage.

## LECTURE 2

# I2C Handle Structure & Configuration Structure

## Lecture Objective

In this lecture, the objective is to create the two most important data structures required for the I2C driver:

- I2C Configuration Structure (I2C\_Config\_t)
- I2C Handle Structure (I2C\_Handle\_t)

These structures are to be defined in **stm32f407xx\_i2c\_driver.h**. They follow the same design pattern used previously for drivers like GPIO and SPI.

## 1. Why Do We Need These Structures?

Every peripheral driver requires a way to:

- Store user-defined configuration parameters.
- Keep track of the peripheral being used.
- Pass all necessary information to driver APIs through a single variable.

Instead of passing multiple arguments to every function, all configuration and peripheral information is grouped into structures.

## 2. I2C Configuration Structure (I2C\_Config\_t)

### Purpose

The configuration structure stores all the parameters that the user application can configure before initializing the I2C peripheral. These values are later used by the initialization API (I2C\_Init()).

### Structure Members

```
typedef struct
{
    uint32_t I2C_SCLSpeed;
    uint8_t  I2C_DeviceAddress;
    uint8_t  I2C_ACKControl;
    uint8_t  I2C_FMDutyCycle;

} I2C_Config_t;
```

### Member Description

| Member       | Data Type | Purpose                                                     |
|--------------|-----------|-------------------------------------------------------------|
| I2C_SCLSpeed | uint32_t  | Sets the Serial Clock (SCL) speed (e.g., 100 kHz, 400 kHz). |

| Member            | Data Type | Purpose                                                        |
|-------------------|-----------|----------------------------------------------------------------|
| I2C_DeviceAddress | uint8_t   | Specifies the MCU's own address when acting as an I2C slave.   |
| I2C_ACKControl    | uint8_t   | Enables or disables automatic ACK generation.                  |
| I2C_FMDutyCycle   | uint8_t   | Selects the duty cycle when operating in Fast Mode (>100 kHz). |

### Role of Each Configuration Parameter

- SCL Speed** — Determines the communication speed. Examples: 100 kHz (Standard Mode), 400 kHz (Fast Mode).
- Device Address** — Required when the MCU operates as an I2C slave. The peripheral uses this address to recognize when it is being addressed by a master. Example: 0x68.
- ACK Control** — Controls whether the peripheral automatically sends an ACK after receiving a byte. Possible values: Enable ACK, Disable ACK.
- Fast Mode Duty Cycle** — Used only when SCL Speed > 100 kHz. Determines the high/low ratio of the SCL clock in Fast Mode.

## 3. I2C Handle Structure (I2C\_Handle\_t)

### Purpose

The handle structure combines the peripheral's register address and the user configuration. This handle is passed to almost every driver function, allowing the driver to know which I2C peripheral to operate on and how it should be configured.

### Structure

```
typedef struct
{
    I2C_RegDef_t *pI2Cx;
    I2C_Config_t I2C_Config;
} I2C_Handle_t;
```

### Member Description

| Member     | Purpose                                                                 |
|------------|-------------------------------------------------------------------------|
| pI2Cx      | Pointer to the selected I2C peripheral registers (I2C1, I2C2, or I2C3). |
| I2C_Config | Stores all user-configurable I2C settings.                              |

## 4. Understanding the Handle Structure

### A. Peripheral Pointer

**pI2Cx** points to the register definition structure of the selected peripheral.

```
handle.pI2Cx = I2C1;
```

Now all driver functions know that they are working with I2C1.

## B. Configuration Information

**I2C\_Config** contains all initialization parameters.

```
handle.I2C_Config.I2C_SCLSpeed = I2C_SCL_SPEED_SM;
```

## 5. Relationship Between the Structures



## 6. How the Driver Uses These Structures

### Step 1 – Create a Handle

```
I2C_Handle_t I2CHandle;
```

### Step 2 – Select the Peripheral

```
I2CHandle.pI2Cx = I2C1;
```

### Step 3 – Fill Configuration

```
I2CHandle.I2C_Config.I2C_SCLSpeed = 1000000;
I2CHandle.I2C_Config.I2C_DeviceAddress = 0x68;
I2CHandle.I2C_Config.I2C_ACKControl = ENABLE;
I2CHandle.I2C_Config.I2C_FMDutyCycle = DUTY_2;
```

### Step 4 – Initialize

```
I2C_Init(&I2CHandle);
```

The initialization function reads all these values from the handle and configures the hardware registers accordingly.

## 7. File Location

Both structures should be declared in:

```
Drivers/  
+-- stm32f407xx_i2c_driver.h
```

The corresponding implementation (I2C\_Init(), I2C\_MasterSendData(), etc.) will use these structures from the .c file.

## 8. Comparison with Previous Drivers

| Driver | Configuration Structure | Handle Structure |
|--------|-------------------------|------------------|
| GPIO   | GPIO_PinConfig_t        | GPIO_Handle_t    |
| SPI    | SPI_Config_t            | SPI_Handle_t     |
| I2C    | I2C_Config_t            | I2C_Handle_t     |

The design is consistent across all peripheral drivers, making the driver framework modular and reusable.

### KEY TAKEAWAYS

- Define two structures in stm32f407xx\_i2c\_driver.h: I2C\_Config\_t (user-configurable settings) and I2C\_Handle\_t (peripheral pointer + configuration).
- I2C\_Config\_t includes: SCL Speed, Device Address, ACK Control, Fast Mode Duty Cycle.
- I2C\_Handle\_t includes: pointer to the I2C peripheral registers (pI2Cx) and an instance of I2C\_Config\_t.
- The handle structure is passed to driver APIs (such as I2C\_Init()) so the driver knows which peripheral to configure and how to configure it.

## LECTURE 3

# Configuration Macros for User Options

## Lecture Objective

---

In this lecture, the goal is to:

- Include the required header files.
- Link the I2C driver with the device-specific header.
- Create user configuration macros for the I2C configuration structure.
- Define options for SCL Speed, ACK Control, and Fast Mode Duty Cycle.

These macros allow the user to configure the I2C peripheral in a simple and readable way.

## 1. Include Required Header Files

---

*In stm32f407xx\_i2c\_driver.h*

```
#include "stm32f407xx.h"
```

**Purpose:** This provides access to register definitions, peripheral base addresses, bit position macros, clock control macros, and standard data types.

## 2. Include the Driver Header in the MCU Header

---

At the end of the device-specific header file, **stm32f407xx.h**, include the I2C driver header.

```
#include "stm32f407xx_i2c_driver.h"
```

**Purpose:** This makes the driver declarations available throughout the project.

## 3. stm32f407xx\_i2c\_driver.c

---

At this stage, the source file is still empty. Implementation of the APIs will begin in later lectures.

## 4. Creating Configuration Macros

---

Instead of using raw numbers throughout the program, meaningful macros are defined.

**Benefits:**

- Improves readability
- Makes code easier to maintain

- Prevents magic numbers
- Simplifies user configuration

## 5. SCL Speed Options

The first configurable parameter is the Serial Clock (SCL) speed. The user selects one of the predefined speed macros.

### Standard Mode (SM)

Standard Mode supports 100 kHz.

```
#define I2C_SCL_SPEED_SM    100000U
```

### Fast Mode (FM)

Fast Mode supports speeds greater than 100 kHz. For this driver, three speed options are created.

```
#define I2C_SCL_SPEED_SM    100000U    // 100 kHz (Standard Mode)
#define I2C_SCL_SPEED_FM2K  200000U    // 200 kHz (Fast Mode)
#define I2C_SCL_SPEED_FM4K  400000U    // 400 kHz (Fast Mode)
```

### Speed Summary

| Macro              | Speed   | Mode          |
|--------------------|---------|---------------|
| I2C_SCL_SPEED_SM   | 100 kHz | Standard Mode |
| I2C_SCL_SPEED_FM2K | 200 kHz | Fast Mode     |
| I2C_SCL_SPEED_FM4K | 400 kHz | Fast Mode     |

Any I2C speed greater than 100 kHz is considered Fast Mode.

## 6. Device Address

The device address is not assigned through predefined macros. Instead, it is supplied directly by the application.

```
I2CHandle.I2C_Config.I2C_DeviceAddress = 0x68;
```

### Why?

- Every slave device has a unique 7-bit address.
- The address depends on the specific hardware being used.
- Therefore, fixed macros are not practical.

**Device Address Width:** The STM32 I2C peripheral uses a 7-bit slave address. Although stored in an 8-bit (uint8\_t) variable, only the lower 7 bits are used.

## 7. ACK Control Options

The next configurable item is ACK Control. Two macros are defined:

```
#define I2C_ACK_ENABLE      1
#define I2C_ACK_DISABLE    0
```

**Purpose:** These macros control whether the I2C peripheral automatically acknowledges received bytes.

**Hardware Register:** ACK is controlled by the **CR1** register, bit **ACK (Bit 10)**.

| Value | Meaning      |
|-------|--------------|
| 0     | ACK Disabled |
| 1     | ACK Enabled  |

**Important default behavior:** By default, the STM32 I2C peripheral has automatic acknowledgment disabled. No ACK is sent after receiving a byte unless the ACK bit is enabled. This must be configured by software if ACK is required.

## 8. Fast Mode Duty Cycle Options

Duty cycle is applicable only in Fast Mode. Two options are available.

**Duty Cycle = 2**

```
#define I2C_FM_DUTY_2      0
```

**Duty Cycle = 16/9**

```
#define I2C_FM_DUTY_16_9  1
```

**Hardware Register:** Duty cycle is configured through the **CCR (Clock Control Register)**, field **DUTY**.

| DUTY Bit | Duty Cycle |
|----------|------------|
| 0        | 2          |
| 1        | 16/9       |

The instructor mentions that the concept of duty cycle will be explained in detail in a later lecture. For now, only the macros are created.

## 9. Summary of User Configuration Macros

```
/*=====
 * I2C_SCL Speed
 *=====*/
#define I2C_SCL_SPEED_SM      100000U
#define I2C_SCL_SPEED_FM2K    200000U
#define I2C_SCL_SPEED_FM4K    400000U

/*=====
 * ACK Control
 *=====*/
#define I2C_ACK_ENABLE        1
#define I2C_ACK_DISABLE      0

/*=====
 * Fast Mode Duty Cycle
 *=====*/
#define I2C_FM_DUTY_2         0
#define I2C_FM_DUTY_16_9     1
```

## 10. How These Macros Are Used

```
I2C_Handle_t I2CHandle;

I2CHandle.I2C_Config.I2C_SCLSpeed = I2C_SCL_SPEED_SM;
I2CHandle.I2C_Config.I2C_DeviceAddress = 0x68;
I2CHandle.I2C_Config.I2C_ACKControl = I2C_ACK_ENABLE;
I2CHandle.I2C_Config.I2C_FMDutyCycle = I2C_FM_DUTY_2;
```

This makes the initialization code more readable than using raw numeric values.

## 11. Configuration Flow

```
User Application
  |
  v
Select Speed Macro
  |
  v
Select ACK Option
  |
  v
Select Duty Cycle
  |
  v
Enter Device Address
  |
  v
Fill I2C_Config Structure
  |
  v
Pass Handle to I2C_Init()
```

## File Organization

```
Drivers/
|
+-- stm32f407xx_i2c_driver.h
|   +-- Configuration Structure
|   +-- Handle Structure
|   +-- Speed Macros
|   +-- ACK Macros
|   +-- Duty Cycle Macros
|
+-- stm32f407xx_i2c_driver.c
|   +-- (Empty for now)
|
+-- stm32f407xx.h
    +-- Includes stm32f407xx_i2c_driver.h
```

### KEY TAKEAWAYS

- Include stm32f407xx.h in stm32f407xx\_i2c\_driver.h, and include stm32f407xx\_i2c\_driver.h at the end of stm32f407xx.h.
- Create macros for configurable options instead of using magic numbers.
- SCL speed macros: I2C\_SCL\_SPEED\_SM (100 kHz), I2C\_SCL\_SPEED\_FM2K (200 kHz), I2C\_SCL\_SPEED\_FM4K (400 kHz).
- ACK control macros: I2C\_ACK\_ENABLE = 1, I2C\_ACK\_DISABLE = 0. Controlled by Bit 10 (ACK) of CR1, disabled by default.
- Fast Mode duty cycle macros: I2C\_FM\_DUTY\_2 = 0, I2C\_FM\_DUTY\_16\_9 = 1.
- The slave device address is supplied directly by the user and does not require predefined macros.

## LECTURE 4

# Creating API Prototypes

### Lecture Objective

- Create the API prototypes for the I2C driver.
- Reuse the common driver framework developed for the SPI driver.
- Identify which APIs can be directly reused, modified, removed, or implemented later.
- Assign a few APIs as implementation exercises.

### 1. Reusing the SPI Driver Framework

Instead of writing everything from scratch, the instructor copies the API section from **stm32f407xx\_spi\_driver.h** and pastes it into **stm32f407xx\_i2c\_driver.h**. Then, all occurrences of **SPI** are replaced with **I2C**.

This saves development time because most peripheral drivers follow a common architecture.

### 2. Common Driver APIs

Most STM32 peripheral drivers provide similar categories of APIs:

- Clock control
- Initialization
- De-initialization
- Peripheral control
- Interrupt configuration
- Flag status
- Callback functions

Only the communication-specific APIs differ between peripherals.

### 3. Peripheral Clock Control API

```
void I2C_PeriClockControl(I2C_RegDef_t *pI2Cx, uint8_t EnorDi);
```

**Purpose:** Enables or disables the clock for the selected I2C peripheral.

| Parameter | Description                    |
|-----------|--------------------------------|
| pI2Cx     | Pointer to I2C1, I2C2, or I2C3 |

| Parameter | Description                        |
|-----------|------------------------------------|
| EnorDi    | Enable or Disable peripheral clock |

## 4. Initialization API

```
void I2C_Init(I2C_Handle_t *pI2CHandle);
```

**Purpose:** Configures the I2C peripheral using the values stored in the handle structure. This function will configure: SCL Speed, Device Address, ACK Control, Duty Cycle.

This API will be implemented in a later lecture.

## 5. De-Initialization API

```
void I2C_DeInit(I2C_RegDef_t *pI2Cx);
```

**Purpose:** Resets the selected I2C peripheral to its default hardware state. This API is simple and similar to the SPI de-initialization function.

**Exercise:** Implement this function.

## 6. Communication APIs

Unlike SPI, the send/receive APIs cannot simply be copied because I2C communication is different. Therefore, the SPI APIs are removed, and dedicated I2C communication APIs will be created later, including separate APIs for Master Transmission, Master Reception, Slave Transmission, and Slave Reception.

This separation is required because only the master generates the START condition and initiates communication.

## 7. Interrupt Configuration APIs

These APIs are identical to those used in GPIO and SPI drivers.

### *Interrupt Configuration*

```
void I2C_IRQInterruptConfig(uint8_t IRQNumber, uint8_t EnorDi);
```

**Purpose:** Enables or disables a specific I2C interrupt.

### *Interrupt Priority Configuration*

```
void I2C_IRQPriorityConfig(uint8_t IRQNumber, uint32_t IRQPriority);
```

**Purpose:** Assigns a priority level to the specified I2C interrupt.

**Implementation:** Reuse the existing GPIO/SPI code; no changes are required.

## 8. IRQ Handling Function

The SPI interrupt handler is not copied. Reason: the I2C interrupt mechanism is more complex and differs from SPI. Therefore, the I2C interrupt handler will be designed separately in a later lecture.

## 9. Peripheral Control API

```
void I2C_PeripheralControl(I2C_RegDef_t *pI2Cx, uint8_t EnOrDi);
```

**Purpose:** Enables or disables the I2C peripheral after its clock has been enabled. This is different from clock control:

- **Clock Control:** Supplies the clock to the peripheral.
- **Peripheral Control:** Activates or deactivates the I2C peripheral itself.

**Hardware Register:** Peripheral control is performed using the CR1 register, field **PE (Peripheral Enable)**.

| Value | Meaning             |
|-------|---------------------|
| 0     | Peripheral Disabled |
| 1     | Peripheral Enabled  |

The driver will set or clear this bit depending on the EnOrDi parameter.

**Exercise:** Implement this function.

## 10. Flag Status API

```
uint8_t I2C_GetFlagStatus(I2C_RegDef_t *pI2Cx, uint32_t FlagName);
```

**Purpose:** Checks whether a specific I2C status flag is set. Examples of I2C flags include:

- TXE (Transmit Data Register Empty)
- RXNE (Receive Data Register Not Empty)
- BUSY
- BTF (Byte Transfer Finished)

The detailed use of these flags will be covered later.

## 11. Application Callback

```
void I2C_ApplicationEventCallback(I2C_Handle_t *pI2CHandle, uint8_t AppEv);
```

**Purpose:** Provides a callback mechanism for interrupt-driven communication. When an interrupt event occurs (such as transmission complete or reception complete), the driver can notify the application through this callback.

This API will be used when interrupt-based communication is implemented.

## 12. APIs Removed from the SPI Driver

| SPI API           | Reason                                 |
|-------------------|----------------------------------------|
| SPI_SendData()    | Replaced by Master/Slave transmit APIs |
| SPI_ReceiveData() | Replaced by Master/Slave receive APIs  |
| SPI IRQ Handler   | I2C interrupt handling is different    |

## 13. APIs to Implement Later

- I2C\_Init()
- Master Transmit API
- Master Receive API
- Slave Transmit API
- Slave Receive API
- I2C Interrupt Handler

## 14. Programming Exercises

The instructor recommends implementing the following APIs as practice.

### 1. Peripheral Clock Control

```
void I2C_PeriClockControl(...);
```

- Similar to SPI
- Enable/Disable RCC clock

### 2. De-Initialization

```
void I2C_DeInit(...);
```

- Similar to SPI
- Reset the peripheral

### 3. Interrupt Configuration

```
void I2C_IRQInterruptConfig(...);
```

- Reuse existing SPI/GPIO code

### 4. Interrupt Priority Configuration

```
void I2C_IRQPriorityConfig(...);
```

- Same implementation as SPI/GPIO

### 5. Peripheral Control

```
void I2C_PeripheralControl(...);
```

- Set/Clear the PE bit in the CR1 register

## 15. API Summary

| API                            | Purpose                                | Status   |
|--------------------------------|----------------------------------------|----------|
| I2C_PeriClockControl()         | Enable/Disable I2C peripheral clock    | Exercise |
| I2C_Init()                     | Configure I2C peripheral               | Later    |
| I2C_DeInit()                   | Reset I2C peripheral                   | Exercise |
| I2C_IRQInterruptConfig()       | Enable/Disable interrupts              | Exercise |
| I2C_IRQPriorityConfig()        | Configure interrupt priority           | Exercise |
| I2C_PeripheralControl()        | Enable/Disable I2C peripheral          | Exercise |
| I2C_GetFlagStatus()            | Read I2C status flags                  | Later    |
| I2C_ApplicationEventCallback() | Notify application of interrupt events | Later    |

## Driver Development Flow

```
Copy SPI Driver API Section
  |
  v
Replace "SPI" with "I2C"
  |
  v
Remove SPI Communication APIs
  |
  v
Add I2C-Specific Communication APIs (Later)
  |
  v
Keep Common Driver APIs
  |
  v
Implement Easy APIs
  |
  v
Implement Initialization & Communication APIs
```

## KEY TAKEAWAYS

- The I2C driver follows the same overall structure as the SPI driver, allowing common APIs to be reused.
- Copy the API prototypes from `stm32f407xx_spi_driver.h` into `stm32f407xx_i2c_driver.h`, then replace SPI with I2C.
- Keep the common APIs: `I2C_PeriClockControl()`, `I2C_Init()`, `I2C_DeInit()`, `I2C_IRQInterruptConfig()`, `I2C_IRQPriorityConfig()`, `I2C_PeripheralControl()`, `I2C_GetFlagStatus()`, `I2C_ApplicationEventCallback()`.
- Remove the SPI send/receive APIs because I2C requires separate Master TX, Master RX, Slave TX, and Slave RX APIs.
- The PE (Peripheral Enable) bit in the CR1 register is used to enable or disable the I2C peripheral.
- As practice, implement the simpler APIs (`PeriClockControl`, `DeInit`, `IRQInterruptConfig`, `IRQPriorityConfig`, `PeripheralControl`) before the more complex initialization and communication functions.

## LECTURE 5

# Introduction to I2C\_Init() API

## Lecture Objective

---

- Begin implementing the I2C\_Init() API.
- Understand the sequence of steps required to initialize an I2C peripheral.
- Learn why the peripheral must remain disabled during configuration.
- Prepare for the upcoming discussion on serial clock (SCL) configuration.

## 1. Importance of I2C\_Init()

---

The I2C\_Init() function is responsible for configuring the I2C peripheral according to the settings provided by the user in the I2C\_Config\_t structure. Typical responsibilities include:

- Selecting the I2C operating mode.
- Configuring the SCL clock speed.
- Setting the device (own) address.
- Enabling or disabling ACK.
- Configuring the rise time.
- Enabling the peripheral after all settings are complete.

## 2. Prerequisite: Enable the Peripheral Clock

---

Before configuring any I2C registers, the RCC clock for the selected I2C peripheral must be enabled.

```
I2C_PeriClockControl(I2C1, ENABLE);
```

Clock enable is mandatory for all STM32 peripherals before accessing their registers.

## 3. I2C Initialization Steps

---

### *Step 1 – Configure the I2C Operating Mode*

The peripheral can operate in different speed modes: Standard Mode, Fast Mode, Fast Mode Plus, Ultra Fast Mode (supported by some devices). The selected mode determines how the hardware generates the I2C clock and timing.

In this driver, the focus is on Standard Mode and Fast Mode.

### Step 2 – Configure the Serial Clock (SCL) Speed

The next step is to select the desired SCL frequency. Examples: 50 kHz, 100 kHz, 200 kHz, 400 kHz. This value is taken from **I2C\_Config.I2C\_SCLSpeed**.

#### Effect of Clock Speed on Bus Length

Higher clock frequencies require shorter bus lengths because signal integrity degrades more quickly over long wires. Lower clock frequencies allow longer cable lengths.

| SCL Speed               | Typical Bus Length            |
|-------------------------|-------------------------------|
| Lower (e.g., 25-50 kHz) | Longer wires can be used      |
| Higher (e.g., 400 kHz)  | Shorter wires are recommended |

This trade-off is important when designing I2C-based hardware systems.

### Step 3 – Configure the Device (Own) Address

If the STM32 acts as an I2C slave, it must be assigned a unique address. This address is stored in **I2C\_Config.I2C\_DeviceAddress**.

The own address is mainly relevant in slave mode. A master initiates communication and does not require an own address for basic operation.

### Step 4 – Configure ACK (Acknowledgement)

ACK is one of the key features of the I2C protocol. The initialization function must configure whether automatic acknowledgement is Enabled or Disabled. The setting comes from **I2C\_Config.I2C\_ACKControl**.

**Important:** On the STM32 I2C peripheral, ACK is disabled by default, so software must enable it if required. This default behavior may differ on other microcontrollers, so always consult the reference manual.

### Step 5 – Configure Rise Time (TRISE)

The final initialization step is to configure the rise time of the I2C signals (SCL and SDA).

#### What is Rise Time?

Rise time is the time taken for a digital signal to transition from LOW (Ground) to HIGH (VCC). It is also related to the slew rate of the signal.

The TRISE register must be configured according to the I2C specification to ensure reliable communication. The detailed calculation and configuration of TRISE will be covered in a later lecture.

## 4. Peripheral Must Be Disabled During Initialization

A key requirement is that all initialization settings must be applied while the I2C peripheral is disabled. The recommended sequence is:

- 1 Peripheral clock enabled.

- 2 Peripheral remains disabled (default state).
- 3 Configure all registers.
- 4 Enable the peripheral using the PE (Peripheral Enable) bit in CR1.

This prevents undefined behavior while modifying configuration registers.

## 5. Beginning the Implementation

---

The instructor starts implementing the following functions in **stm32f407xx\_i2c\_driver.c**:

```
void I2C_Init(I2C_Handle_t *pI2CHandle);  
  
void I2C_DeInit(I2C_RegDef_t *pI2Cx);
```

- I2C\_Init() will be implemented gradually as each configuration topic is explained.
- I2C\_DeInit() is left as an exercise, as it only requires resetting the peripheral through the RCC reset registers.

## 6. De-Initialization Reminder

---

To implement **void I2C\_DeInit(I2C\_RegDef\_t \*pI2Cx);**, refer to the RCC reset registers. The implementation is similar to other STM32 peripheral drivers (e.g., GPIO and SPI).

## 7. Preparing for Serial Clock Configuration

---

Before completing I2C\_Init(), it is necessary to understand how the I2C serial clock is generated. The instructor therefore postpones the implementation and recommends studying two important registers: **CR2 (Control Register 2)** and **CCR (Clock Control Register)**.

| Register | Field | Purpose                                                        |
|----------|-------|----------------------------------------------------------------|
| CR2      | FREQ  | Specifies the APB1 clock frequency used by the I2C peripheral. |
| CCR      | CCR   | Determines the generated SCL clock frequency.                  |

Understanding these fields is essential before writing the clock configuration code.

## Initialization Workflow

---

```
Enable RCC Clock
  |
  v
Keep I2C Peripheral Disabled
  |
  v
Select I2C Mode
  |
  v
Configure SCL Speed
  |
  v
Configure Own Address
  |
  v
Configure ACK
  |
  v
Configure Rise Time (TRISE)
  |
  v
Enable I2C Peripheral (PE Bit)
```

## KEY TAKEAWAYS

- I2C\_Init() configures the I2C peripheral using the parameters stored in the I2C\_Config\_t structure.
- Initialization sequence: enable clock, keep peripheral disabled, configure mode, SCL speed, own address, ACK, rise time (TRISE), then enable via the PE bit in CR1.
- Higher SCL frequencies (e.g., 400 kHz) require shorter bus lengths, while lower frequencies allow longer cables.
- The ACK feature is disabled by default on the STM32 I2C peripheral.
- Before implementing the clock configuration code, study the CR2 (FREQ) and CCR fields, as they determine how the I2C serial clock is generated.

## LECTURE 6

# Serial Clock (SCL) Configuration using CR2 and CCR

## Lecture Objective

In this lecture, the objective is to understand how the STM32F4 I2C peripheral generates the desired Serial Clock (SCL) frequency. The lecture explains:

- Which registers control the I2C clock.
- How to configure the CR2 and CCR registers.
- How to calculate the CCR value.
- The difference between Standard Mode and Fast Mode.
- How the Duty Cycle affects Fast Mode operation.

## 1. How is the I2C Serial Clock Generated?

The I2C peripheral does not generate the SCL frequency directly. Instead, it derives the SCL clock from the APB1 peripheral clock (PCLK1). The user must configure two registers: **CR2** and **CCR**. These registers determine:

- Serial Clock (SCL) frequency
- Setup time
- Hold time
- Other I2C timing parameters

## 2. Registers Used for Clock Configuration

| Register | Important Field | Purpose                                       |
|----------|-----------------|-----------------------------------------------|
| CR2      | FREQ            | Stores the APB1 clock frequency (in MHz).     |
| CCR      | CCR, FS, DUTY   | Controls SCL frequency, mode, and duty cycle. |

## 3. CR2 Register – FREQ Field

### *Purpose*

The FREQ field tells the I2C peripheral the frequency of the clock supplied from the APB1 bus (PCLK1). The I2C hardware uses this information to calculate all timing parameters internally.

**Important:** The value written is not the frequency in Hz, but the frequency in MHz.

| APB1 Clock | Value Written to FREQ |
|------------|-----------------------|
| 8 MHz      | 8                     |
| 16 MHz     | 16                    |
| 24 MHz     | 24                    |
| 42 MHz     | 42                    |

**Example:** Suppose PCLK1 = 16 MHz. Then: **CR2.FREQ = 16**. This value is mandatory because the I2C peripheral uses it for internal timing calculations.

## 4. Standard Mode Example

### Given

| Parameter   | Value         |
|-------------|---------------|
| Mode        | Standard Mode |
| Desired SCL | 100 kHz       |
| PCLK1       | 16 MHz        |

### Step 1 – Select Standard Mode

In the CCR register, configure the FS (Fast/Standard Mode) bit.

| FS Bit | Mode          |
|--------|---------------|
| 0      | Standard Mode |
| 1      | Fast Mode     |

For this example: **FS = 0**

### Step 2 – Configure CR2.FREQ

Since PCLK1 = 16 MHz, write: **CR2.FREQ = 16**

### Step 3 – Calculate CCR Value

For Standard Mode, the reference manual provides (assuming a 50% duty cycle,  $T_{\text{HIGH}} = T_{\text{LOW}}$ ):

$$T_{\text{HIGH}} = \text{CCR} \times T_{\text{PCLK1}}$$

**Calculate Clock Period** — Desired SCL: 100 kHz

$$T = 1 / 100000 = 10 \mu\text{s}$$

Since duty cycle is 50%:

$$T_{\text{HIGH}} = 5 \mu\text{s} \quad T_{\text{LOW}} = 5 \mu\text{s}$$

**Calculate APB1 Clock Period** — Given PCLK1 = 16 MHz:

$$T_{\text{PCLK1}} = 1 / 16 \text{ MHz} = 62.5 \text{ ns}$$

**Compute CCR:**

$$\text{CCR} = T_{\text{HIGH}} / T_{\text{PCLK1}} = 5 \mu\text{s} / 62.5 \text{ ns} = 80$$

Therefore: **CCR = 80** (hexadecimal: **0x50**)

### *Standard Mode Register Settings*

| Register | Value |
|----------|-------|
| CR2.FREQ | 16    |
| CCR.CCR  | 80    |
| CCR.FS   | 0     |

This generates: **SCL = 100 kHz**

## 5. Fast Mode Example

*Given*

| Parameter   | Value     |
|-------------|-----------|
| Mode        | Fast Mode |
| Desired SCL | 200 kHz   |
| PCLK1       | 16 MHz    |

**Step 1 – Select Fast Mode**

$$\text{CCR.FS} = 1$$

**Step 2 – Select Duty Cycle**

Fast Mode requires choosing one of two duty cycles. This is done using the DUTY bit in the CCR register.

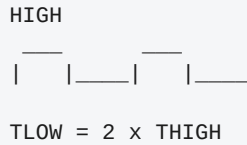
## 6. Fast Mode Duty Cycle Options

### Option 1

When **DUTY = 0**, relationship:

$$T_{\text{LOW}} = 2 \times T_{\text{HIGH}}$$

The SCL waveform looks like:



### Option 2

When **DUTY = 1**, relationship:

$$T_{\text{LOW}} = (9/16) \times T_{\text{HIGH}} \quad (\text{approximately } 1.8 \times T_{\text{HIGH}})$$

This option allows the peripheral to reach the maximum Fast Mode speed (400 kHz).

## 7. Fast Mode Timing Formula

For **DUTY = 0**, the total clock period becomes:

$$T_{\text{HIGH}} + T_{\text{LOW}} = 3 \times \text{CCR} \times T_{\text{PCLK1}}$$

**Example Calculation** — Desired frequency: 200 kHz

$$T = 1 / 200000 = 5 \mu\text{s}$$

Using  $T = 3 \times \text{CCR} \times T_{\text{PCLK1}}$ , substituting:

$$\text{CCR} = 5 \mu\text{s} / (3 \times 62.5 \text{ ns})$$

Result: **CCR ≈ 26**

### Register Settings

| Register | Value |
|----------|-------|
| FS       | 1     |
| DUTY     | 0     |
| CR2.FREQ | 16    |

| Register | Value |
|----------|-------|
| CCR      | 26    |

## 8. If DUTY = 1

The reference manual provides a different equation. Instead of multiplying by 3, the divider becomes **25**. The calculation therefore changes accordingly.

This configuration is typically used when operating at 400 kHz, the maximum Fast Mode speed.

## 9. I2C Timing Requirements

The I2C specification defines minimum HIGH and LOW times for the SCL clock.

### *Standard Mode*

| Parameter | Minimum     |
|-----------|-------------|
| TLOW      | 4.7 $\mu$ s |
| THIGH     | 4.0 $\mu$ s |

### *Fast Mode*

| Parameter | Minimum     |
|-----------|-------------|
| TLOW      | 1.3 $\mu$ s |
| THIGH     | 0.6 $\mu$ s |

The selected CCR value and duty cycle must satisfy these timing requirements.

### *Why 100 kHz with 50% Duty Cycle Works*

At 100 kHz, clock period = 10  $\mu$ s. Therefore  $T_{\text{HIGH}} = 5 \mu\text{s}$ ,  $T_{\text{LOW}} = 5 \mu\text{s}$ .

| Requirement                 | Actual          | Result |
|-----------------------------|-----------------|--------|
| THIGH $\geq 4 \mu\text{s}$  | 5 $\mu\text{s}$ | OK     |
| TLOW $\geq 4.7 \mu\text{s}$ | 5 $\mu\text{s}$ | OK     |

Thus, the generated waveform satisfies the I2C specification.

## 10. Summary of Register Configuration

### *Standard Mode*

FS = 0  
CR2.FREQ = PCLK1 (MHz)  
CCR = Calculated Value

### ***Fast Mode***

FS = 1  
Select DUTY  
CR2.FREQ = PCLK1 (MHz)  
CCR = Calculated Value

## **Configuration Flow**

```
graph TD
    A[Enable I2C Clock] --> B[Determine PCLK1]
    B --> C[Write PCLK1(MHz) to CR2.FREQ]
    C --> D[Select Mode (Standard/Fast)]
    D --> E[If Fast Mode: Select DUTY Cycle]
    E --> F[Calculate CCR]
    F --> G[Program CCR Register]
    G --> H[Desired SCL Generated]
```

## **Important Formulas**

### **APB Clock Period**

$$T_{\text{PCLK1}} = 1 / \text{PCLK1}$$

### **Standard Mode**

$$\text{CCR} = T_{\text{HIGH}} / T_{\text{PCLK1}}$$

For a 50% duty cycle:

$$T_{\text{HIGH}} = T_{\text{SCL}} / 2$$

**Fast Mode (DUTY = 0)**

$$T_{\text{SCL}} = 3 \times \text{CCR} \times T_{\text{PCLK1}}$$

**Fast Mode (DUTY = 1)**

$$T_{\text{SCL}} = 25 \times \text{CCR} \times T_{\text{PCLK1}}$$

## KEY TAKEAWAYS

- The I2C SCL clock is derived from the APB1 clock (PCLK1) and is configured using the CR2 and CCR registers.
- The CR2.FREQ field must be programmed with the APB1 clock frequency in MHz (e.g., write 16 for a 16 MHz PCLK1).
- CCR register controls: FS bit (Standard/Fast Mode), DUTY bit (Fast Mode duty cycle), and the CCR field (clock divider).
- Standard Mode (100 kHz), PCLK1 = 16 MHz: CR2.FREQ = 16, FS = 0, CCR = 80 (0x50).
- Fast Mode (200 kHz), PCLK1 = 16 MHz, DUTY = 0: CR2.FREQ = 16, FS = 1, DUTY = 0, CCR ≈ 26.
- Fast Mode duty cycle options: DUTY=0 gives TLOW = 2×THIGH; DUTY=1 gives TLOW = (9/16)×THIGH, typically used for 400 kHz.
- The generated SCL waveform must satisfy the minimum HIGH and LOW timing requirements defined by the I2C specification.

## LECTURE 7

# I2C Protocol – Clock Stretching

### Lecture Objective

---

- What Clock Stretching is.
- Why it is required in I2C communication.
- How a slave temporarily pauses communication.
- How clock stretching prevents communication errors.
- How STM32 hardware automatically handles clock stretching.

### 1. What is Clock Stretching?

---

Clock Stretching is one of the most important features of the I2C protocol.

#### *Definition*

Clock stretching is the process of holding the Serial Clock (SCL) line LOW for a period of time. While SCL remains LOW:

- The clock cannot continue.
- The entire I2C communication is temporarily paused.
- Communication resumes only after the device releases the clock line.

**Simple Definition:** Clock stretching is a mechanism that allows an I2C device (usually the slave) to temporarily pause data transfer by keeping the SCL line LOW.

### 2. Why is Clock Stretching Needed?

---

Normally, the master controls the clock speed in an I2C bus. However, there are situations where the slave cannot process data as quickly as the master is sending it. Examples:

- Processing received data.
- Reading from internal memory.
- Accessing EEPROM or sensors.
- Performing internal calculations.

Instead of losing data, the slave requests additional time by stretching the clock.

### 3. Who Controls the Clock?

---



- The slave receives the byte.
- The slave is still busy processing the data.
- Instead of allowing the next clock pulse, the slave holds SCL LOW.
- Communication pauses.
- Once ready, the slave releases SCL.
- The 9th clock pulse is generated.
- The slave transmits the ACK.

## 7. Why is Clock Stretching Important?

---

Without clock stretching:

```
Master sends 8 bits
      |
      v
Master immediately generates 9th clock
      |
      v
Slave is not ready
      |
      v
No ACK generated
      |
      v
Master interprets it as NACK
      |
      v
Communication Error
```

The master would incorrectly assume that the slave did not acknowledge the data.

## 8. How Clock Stretching Prevents Errors

---

With clock stretching:

```
Master sends 8 bits
  |
  v
Slave is busy
  |
  v
Slave keeps SCL LOW
  |
  v
Communication Pauses
  |
  v
Slave becomes ready
  |
  v
Slave releases SCL
  |
  v
Master generates 9th clock
  |
  v
Slave sends ACK
```

The transfer continues correctly without data loss.

## 9. Slave's Message to the Master

---

By stretching the clock, the slave is effectively saying:

"I'm busy right now. Please wait before sending the next clock pulse."

The master automatically waits until the slave releases the clock.

## 10. ACK and Clock Stretching

---

Recall that 8 clock cycles are used to transfer one data byte, and the 9th clock cycle is used for the ACK/NACK response.

If the slave is not ready to send the ACK immediately after receiving the byte, it stretches the clock until it can respond.

## 11. Does Software Need to Control Clock Stretching?

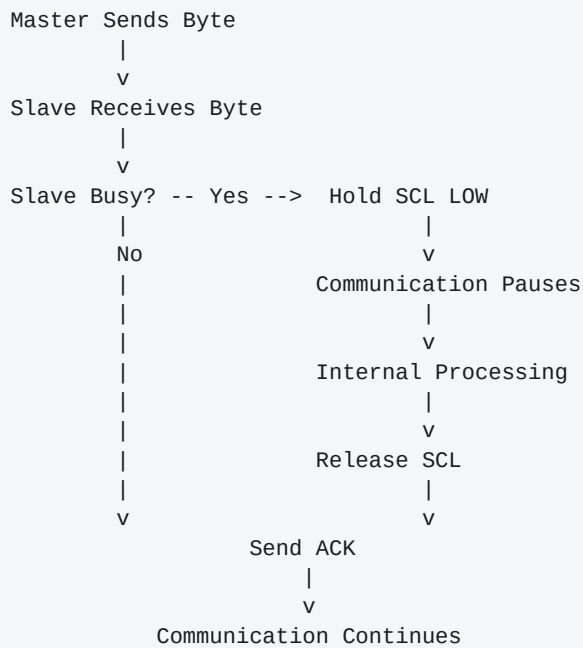
---

**No.** The STM32 I2C hardware manages clock stretching automatically. The programmer only needs to enable the clock stretching feature (if required). The hardware takes care of:

- Holding SCL LOW.
- Releasing SCL.
- Resuming communication.

No manual control of the SCL line is required in software.

## 12. Clock Stretching Timeline



## 13. Advantages of Clock Stretching

- Prevents data loss.
- Allows slower slave devices to communicate with faster masters.
- Eliminates the need for fixed communication delays.
- Ensures reliable data transfer.
- Automatically synchronized by I2C hardware.

## 14. Practical Examples

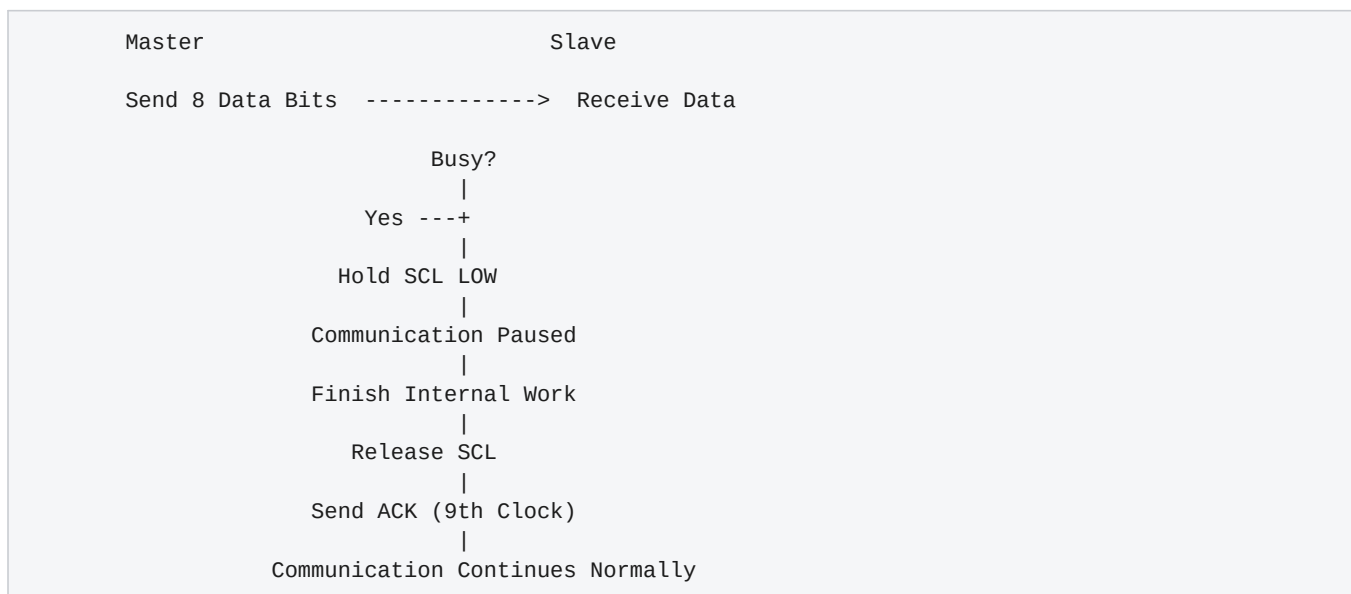
Clock stretching is commonly used by devices such as:

- EEPROMs
- Temperature sensors
- ADCs
- DACs
- Real-Time Clock (RTC) chips
- Other I2C peripherals that require extra processing time before responding

## Key Points to Remember

| Feature                     | Description                                                                                |
|-----------------------------|--------------------------------------------------------------------------------------------|
| <b>Clock Stretching</b>     | Holding the SCL line LOW to pause communication.                                           |
| <b>Controlled By</b>        | Usually the slave device (though some masters may also use it in specific cases).          |
| <b>Purpose</b>              | Gives a slower device extra time to process data.                                          |
| <b>Communication Status</b> | Paused while SCL is LOW.                                                                   |
| <b>Resume Condition</b>     | Communication resumes when the slave releases SCL.                                         |
| <b>Software Role</b>        | Only enable the feature; the hardware automatically performs the stretching and releasing. |

## Summary Diagram



## KEY TAKEAWAYS

- Clock Stretching is an I2C feature that allows a device to pause communication by holding the SCL line LOW.
- The master normally controls the clock, but the slave can delay the next clock pulse if it needs more time to process data.
- Without clock stretching, a busy slave might fail to send an ACK, causing the master to interpret the response as a NACK and resulting in a communication error.
- After completing its internal processing, the slave releases SCL, allowing the master to continue the transfer.
- On STM32 microcontrollers, clock stretching is handled automatically by the I2C hardware; software generally only needs to enable the feature.